

ASSIGNMENT 2E-commerce platform -

This e-commerce platform uses a microservices architecture where each major function is its own service. Events are used to communicate between these services, for example -

a new order creates an event that is consumed by a service to update inventory. SOLID principle enhance the design by ensuring each microservice has a single responsibility and dependencies are managed appropriately, according to Red DRY and KISS principles are applied to minimize code duplication and keep the architecture simple.

microservices and event architecture

- Product catalog service - manages all product related data, including inventory, pricing and description.
- user management service - handles user registration authentication and profile management.
- other services - provide processes orders, interact with payment gateways, and manages shipping information.
- Payment service - handles payment processing including secure transactions and fraud detection.
- Shipping service - manages shipping information coordinates with logistics providers and track order fulfillment.

Events:

- order created event - triggered when a new order is placed, consumed by inventory service, shipping service and payment service
- Product updated event - triggered when a product inventory price or description changes, consumed by product catalog service.

shipping

- Payment Processed event - triggered when a payment is successfully processed, consumed by order service and shipping service
- shipping completed event - triggered when a shipment is completed consumed by order service and potentially customer notifications

SOLID PRINCIPLES

- Single Responsibility Principle
- Open / Closed Principle
- Liskov Substitution Principle
- ~~Integration~~ Interface segregation Principle
- Dependancy Inversion Principle

DRY (Do not Repeat Yourself)

- common logic, like validation rules or data formats, should be encapsulated within a shared ~~memory~~ library or component to avoid duplication.

KISS (Keep it Simple, Stupid)

- Avoid unnecessary complexity use the simplest approach that meets the requirement, making the architecture easier to understand and maintain.